

UNIVERSIDAD NACIONAL DE GENERAL SARMIENTO

Instituto de Ciencias

TRABAJO PRÁCTICO

Programación I

VIDEOJUEGO EN JAVA

INTEGRANTES DEL EQUIPO:

Apellido, Nombre: Cintia Anahi Noli Socci

Legajo: 41804415/2022

Email: Cintianolis@gmail.com

Fecha de Entrega: Junio, 2026

1. Introducción

Este trabajo práctico consiste en el desarrollo de un juego de plataformas en 2D utilizando la librería gráfica entorno. La idea principal es controlar a un personaje que debe avanzar saltando por diferentes islas flotantes hacia la derecha (side-scrolling). El objetivo es esquivar o eliminar a los enemigos disparando con el mouse, agarrar ítems de vida y llegar a un castillo que marca el final del nivel y la victoria.

2. Descripción de las Clases

Juego

Es la clase principal que maneja todo el bucle del juego a través del método tick().

Variables: Controla el entorno, las instancias del personaje, el castillo, el disparo, un ítem y los arreglos para manejar los pisos y los enemigos. También usa booleanos para saber si el jugador ganó o perdió.

Métodos: En el constructor se arma el mapa posicionando las plataformas y el castillo. El método tick() corre en cada instante de tiempo; limpia la pantalla, lee las teclas para mover al personaje, aplica la gravedad, chequea colisiones y dibuja todo. También incluye generarEnemigos(), que se encarga de mantener un máximo de 5 enemigos activos spawnéandolos en posiciones aleatorias sobre las islas.

Personaje

Representa al héroe del juego y se encarga de sus movimientos y físicas.

Variables: Posición (x, y), dimensiones (ancho, alto), velocidad vertical (velocidadY), cantidad de vidas y estados lógicos como si está apoyado (enPiso) y el registro de la posición previa (yAnterior). Además cuenta con constantes para la GRAVEDAD, la fuerza del SALTO y la VELOCIDAD de caminata.

Métodos: Tiene métodos para dibujar(), moverDerecha(), moverIzquierda() y saltar(). Para que interactúe con el entorno se usan tocarPiso(), tocarTecho() y tocarCostado(), que reacomodan la posición si se mete adentro de una plataforma. Por último, limitarPantalla() evita que salga de los bordes laterales y maneja la pérdida de vida si se cae al vacío.

Enemigo

Son los obstáculos móviles que el jugador debe sortear.

Variables: Coordenadas (x, y) y la velocidad que determina hacia qué lado se mueve.

Métodos: mover() actualiza su posición horizontal para que patrulle, moverIzquierda() corre al enemigo al mismo ritmo que la cámara cuando el jugador avanza, dibujar() lo muestra como un círculo rojo y colisiona() detecta si tocó al personaje. fueraPantalla() avisa cuando el enemigo ya no es visible para poder eliminarlo.

Proyectil

Es la munición que dispara el jugador.

Variables: Posición en números decimales (x, y) y las direcciones de desplazamiento en los ejes (dx, dy).

Métodos: En el constructor recibe dónde nace (el personaje) y la posición del mouse para calcular el ángulo y la velocidad del tiro mediante vectores. Tiene métodos para mover(), moverIzquierda() (por el scroll del mapa), dibujar() y colisiona() para ver si impactó en un enemigo.

Piso / Castillo / Item / Dibujar

- **Piso:** Define las plataformas sólidas donde el personaje puede pararse. Tiene variables para su tamaño y posición, y métodos para dibujar() y moverIzquierda().
- **Castillo:** Es la meta final. Tiene variables de tamaño, se dibuja como un rectángulo gris y chequea con colisiona() si el personaje llegó para marcar la victoria.
- **Item:** Es una curación (círculo verde) que aparece a veces al matar un enemigo. Si el personaje colisiona() con él, recupera una vida.
- **Dibujar:** Una clase auxiliar con un método estructurado para facilitar el renderizado de rectángulos a través del entorno.

Problemas Encontrados y Decisiones Tomadas

Scroll de Pantalla (Efecto Cámara)

Problema: Como el escenario es muy largo, el personaje se salía de la ventana por la derecha si avanzaba mucho.

Decisión/Solución: Decidimos que cuando el personaje llegue a la mitad de la pantalla ($x = 400$), su posición horizontal se congela. Si el jugador sigue apretando para avanzar, en lugar de moverse él, se mueven todas las demás cosas (los pisos, los enemigos, los proyectiles, el castillo y el ítem) en sentido contrario. Esto genera la ilusión de que la cámara lo va siguiendo.

Filtramiento de Plataformas (Efecto Traspasar el Suelo)

Problema: Al caer con mucha velocidad por la gravedad, el personaje a veces pasaba de largo el borde del piso en el chequeo de colisión y terminaba atravesando las plataformas.

Decisión/Solución: Agregamos la variable `yAnterior` en el Personaje. Ahora, cuando el juego revisa si está tocando un piso, no sólo mira si se superponen las cajas actuales, sino que verifica si en el instante anterior el personaje venía desde arriba del nivel del suelo. Si es así, se confirma que aterrizó, se frena su velocidad de caída y se lo acomoda exactamente sobre la superficie.

3. Implementación

A continuación se presenta la sección de implementación donde se incluye el código fuente de la clase conductora estructurado desde sus variables de instancia y correctamente comentado con su explicación analítica correspondiente:

Código Fuente (Juego.java)	Explicación Analítica de la Instrucción
<pre>private Personaje personaje; private Piso[] pisos; private Enemigo[] enemigos; private Proyectil disparo; private Castillo castillo; private Item item;</pre>	Aplica el principio de Encapsulamiento restringiendo el acceso externo (private) de las variables de instancia que controlan las entidades del juego.
<pre>private boolean gano = false; private boolean perdio = false;</pre>	Inicializa las variables lógicas (booleanas) encargadas de controlar las pantallas de fin de juego por éxito o derrota.
<pre>Juego() { this.entorno = new Entorno(this, "Proyecto para TP", 800, 600);</pre>	Constructor de la clase. Instancia el objeto motor Entorno configurando el título de la ventana y sus dimensiones de 800x600 píxeles.
<pre> personaje = new Personaje(400, 100, 30, 50);</pre>	Inicializa el objeto del protagonista ubicándolo en el centro horizontal (400) y suspendido en el aire (100) con su escala física correspondiente.
<pre> pisos = new Piso[12]; pisos[0] = new Piso(150, 550, 250, 30); pisos[1] = new Piso(500, 550, 250, 30); // ... (inicialización de los demás pisos del nivel) pisos[11] = new Piso(2600, 520, 300, 30);</pre>	Instancia el arreglo estático para los 12 bloques de suelo flotantes y posiciona geométricamente cada elemento en sus coordenadas espaciales.
<pre> enemigos = new Enemigo[10];</pre>	Crea la estructura del arreglo indexado destinado a contener y administrar las referencias de las amenazas concurrentes en el mapa.
<pre> castillo = new Castillo(2900, 470, 120, 180);</pre>	Instancia la meta del juego en el extremo derecho del escenario (coordenada X = 2900).
<pre> this.entorno.iniciar(); }</pre>	Ejecuta el inicio del motor de ventana, disparando el llamado cíclico periódico del bucle principal.

Código Fuente (Juego.java)	Explicación Analítica de la Instrucción
<pre> @Override public void tick() { if (gano) { entorno.cambiarFont("Arial", 40, Color.GREEN); entorno.escribirTexto("GANASTE", 300, 300); return; } </pre>	Método sobrescrito del ciclo del juego. Si la condición gano es verdadera, interrumpe el juego y muestra el texto de victoria.
<pre> if (perdio) { entorno.cambiarFont("Arial", 40, Color.RED); entorno.escribirTexto("PERDISTE", 300, 300); return; } </pre>	Evalúa el estado de derrota. Si es verdadero, despliega el mensaje de fin de juego en color rojo y detiene la actualización de físicas mediante return.
<pre> entorno.dibujarRectangulo(entorno.anch() / 2, entorno.alto() / 2, entorno.anch(), entorno.alto(), 0, Color.BLACK); </pre>	Limpia el cuadro gráfico renderizando un fondo negro absoluto que cubre la totalidad de la pantalla antes de dibujar el nuevo cuadro.
<pre> if (entorno.estaPresionada('d') entorno.estaPresionada(entorno.TECLA_DERECHA)) { if (personaje.getX() < 400) { personaje.moverDerecha(); } else { for (Piso p : pisos) p.moverIzquierda(5); castillo.moverIzquierda(5); for (Enemigo e : enemigos) if (e != null) e.moverIzquierda(5); if (disparo != null) disparo.moverIzquierda(5); if (item != null) item.moverIzquierda(5); } } </pre>	Mecánica de Scroll Lateral: Si el usuario presiona avanzar, mueve al personaje si está en la mitad izquierda; de lo contrario, desplaza hacia la izquierda de forma inversa todo el escenario dinámico.
<pre> if (entorno.estaPresionada('a') entorno.estaPresionada(entorno.TECLA_IZQUIERDA)) { personaje.moverIzquierda(); } </pre>	Detecta la pulsación de retroceso y desplaza horizontalmente al personaje de forma local hacia la izquierda de la pantalla.
<pre> if (entorno.sePresiono('w') entorno.sePresiono(entorno.TECLA_ARRIBA)) { </pre>	Captura el pulso de la tecla de salto y aplica la fuerza de impulso vertical negativa en la física del protagonista si corresponde.

Código Fuente (Juego.java)	Explicación Analítica de la Instrucción
<pre> personaje.saltar(); } </pre>	
<pre> if (entorno.sePresionoBoton(entorno.BOTON_IZQUIERDO) && disparo == null) { disparo = new Proyectil(personaje.getX(), personaje.getY(), entorno.mouseX(), entorno.mouseY()); } </pre>	Comprueba el clic del mouse y, si no hay un proyectil activo en tránsito, instancia uno nuevo dirigido hacia el puntero.
<pre> personaje.aplicarGravedad(); </pre>	Modifica de manera continua el vector vertical del personaje simulando la aceleración constante por gravedad.
<pre> for (Piso p : pisos) { personaje.tocarPiso(p); personaje.tocarCostado(p); personaje.tocarTecho(p); } </pre>	Lazo iterativo que evalúa las colisiones de las cajas delimitadoras del personaje contra cada una de las plataformas sólidas del nivel.
<pre> personaje.limpiarPantalla(entorno); </pre>	Verifica que el personaje no traspase los bordes de la pantalla y descuenta una vida si detecta una caída al abismo.
<pre> for (Piso p : pisos) p.dibujar(entorno); </pre>	Recorre de forma secuencial la colección indexada de plataformas para dibujarlas en sus posiciones actualizadas.
<pre> generarEnemigos(); </pre>	Invoca al método interno que pobla el mapa con amenazas si la cantidad de oponentes en pantalla es baja.
<pre> for (int i = 0; i < enemigos.length; i++) { if (enemigos[i] != null) { enemigos[i].mover(); enemigos[i].dibujar(entorno); } } </pre>	Actualiza el patrullaje de los enemigos activos y procesa el redibujado de sus elementos gráficos.
<pre> if (enemigos[i].colisiona(personaje)) { personaje.perderVida(); enemigos[i] = null; } </pre>	Evalúa el contacto entre un enemigo y el jugador. Si hay impacto, daña al personaje y destruye la referencia de dicho oponente en el arreglo asignándole null.
<pre> } else if (disparo != null && disparo.colisiona(enemigos[i])) { enemigos[i] = null; disparo = null; if (Math.random() < 0.3) item </pre>	Procesa la colisión del proyectil sobre un rival. Elimina a ambos objetos de pantalla y calcula probabilísticamente (30%) la aparición de un ítem de salud.

Código Fuente (Juego.java)	Explicación Analítica de la Instrucción
<pre>= new Item(personaje.getX(), personaje.getY()); }</pre>	
<pre> } else if (enemigos[i].fueraPantalla()) { enemigos[i] = null; } }</pre>	Elimina de la memoria asignándole el valor null a aquellos oponentes que hayan superado los márgenes útiles del juego.
<pre> if (disparo != null) { disparo.mover(); disparo.dibujar(entorno); if (disparo.fueraPantalla()) disparo = null; }</pre>	Actualiza la trayectoria de la bala activa en el frame actual, la dibuja y la remueve si sale de los límites de la pantalla sin chocar contra nada.
<pre> castillo.dibujar(entorno); if (castillo.colisiona(personaje)) gana = true;</pre>	Renderiza el castillo de la meta y verifica si el héroe interactuó con él para activar el disparador lúdico de la victoria.
<pre> for (int i = 0; i < personaje.getVidas(); i++) { entorno.dibujarCirculo(30 + (i * 40), 30, 15, Color.RED); }</pre>	Dibuja la interfaz de usuario en pantalla (HUD) mostrando de forma lineal los círculos indicadores de salud del personaje.
<pre> if (personaje.getVidas() <= 0) perdio = true;</pre>	Monitorea constantemente la salud del jugador; si el contador llega a cero, activa el estado de derrota.
<pre> if (item != null) { item.dibujar(entorno); if (item.colisiona(personaje)) { personaje.ganarVida(); item = null; } }</pre>	Administra la existencia de la curación: la dibuja en pantalla, evalúa si es absorbida por el héroe para restaurarle salud y la limpia de la memoria.
<pre> personaje.dibujar(entorno); }</pre>	Dibuja al personaje principal sobre la capa final del renderizado de video. Finaliza el bloque operativo de tick().
<pre>private void generarEnemigos() { // ... (lógica interna para calcular el spawn sobre los pisos flotantes) } public static void main(String[] args) { Juego juego = new Juego(); }</pre>	Punto de entrada de la aplicación exigido por la JVM que inicializa el constructor para comenzar la ejecución del programa.

Código Fuente (Juego.java)	Explicación Analítica de la Instrucción
<pre> } } </pre>	

4. Conclusiones

Uso de Objetos: Nos sirvió mucho separar el juego en clases. Que cada objeto (como el Enemigo o el Proyectoil) sepa cómo moverse y dibujarse hizo que el código de la clase Juego sea más limpio y fácil de programar.

Lógica de Colisiones: Lo más difícil de lograr fue que las físicas se sintieran bien. Nos dimos cuenta de que las colisiones en un juego de plataformas no son sólo "si se tocan o no", sino que importa mucho saber desde qué dirección venía el objeto para reaccionar correctamente (saber si pisó el suelo, chocó una pared o pegó en el techo).

Manejo de Memoria: Aprendimos la importancia de limpiar los arreglos. Poner en null los proyectiles o enemigos que mueren o salen de la pantalla es clave para que el juego no consuma recursos de más innecesariamente.